
Report No 1

Team AITEK – Member: Dung

Date 27/11/2025

AITEK

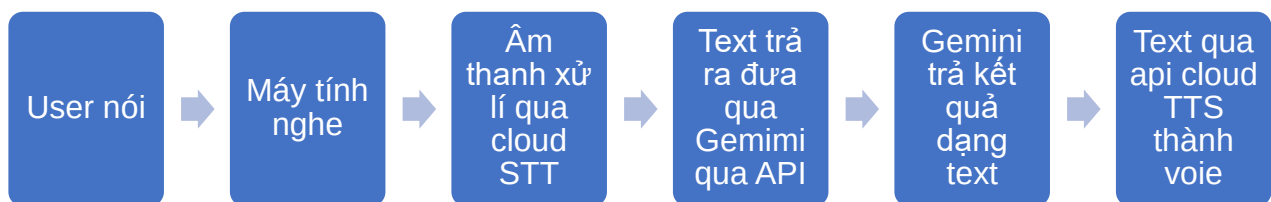
F.A.S.T.
Future Automotive Solution Testing

1. Tóm tắt mục tiêu và tiến độ hiện tại

Mục tiêu đang thực hiện là phát triển một hệ thống trợ lý giọng nói đàm thoại thông minh (Conversational AI Assistant, hiểu đơn giản là chatbot nhưng mà có khả năng nói chuyện cũng được), được thiết kế để tích hợp và lắp vào con MOGI. Hệ thống này sử dụng kiến trúc Hybrid-Cloud/Edge (có thể hiểu là kết hợp khả năng xử lý ngôn ngữ tiên tiến của Cloud API với nền tảng nhúng Raspberry Pi 5) để tạo ra giao diện người dùng tự nhiên và hiệu quả, cho phép người dùng trò chuyện với mogli.

Tính đến thời điểm hiện tại, giai đoạn xây dựng và xác thực hệ thống AI đàm thoại cơ bản đã hoàn thành thử nghiệm các chức năng cơ bản trên laptop cá nhân sử dụng hệ điều hành Windows, sẵn sàng chuyển sang giai đoạn thử nghiệm trên PI5 hoặc phát triển thêm các chức năng khác.

Tóm tắt logic hoạt động của phần mềm hiện tại:



2. Chi tiết tiến độ hiện tại

2.1 Chuẩn bị các Key API của các phần mềm và nguồn cần thiết

- Thông qua Google Studio và Google Cloud Console, nhóm đã lấy được key API của Gemini phiên bản 2.5 Flash, Cloud Speech-to-Text và Cloud Text-to-Speech.
-

- Chuẩn bị sẵn billing account để sử dụng cái API đã kể trên, tuy nhiên Google cung cấp một lượng token hay credits miễn phí nên vấn đề này đã được giải quyết. Nhưng cần cân nhắc nếu request API với số lượng lớn.
- Cài đặt thành công các thư viện cần thiết cho quá trình phát triển chatbot, có thể kể đến như Visual Studio với Dev kit C++, simpleaudio,...

2.2 Cấu trúc code để xử lý lỗi và phát triển mới, các chức năng đã chạy

- Hiện code chat bot được chia làm 2 phần chính, 1 file chatbot_main.py, chứa lệnh sử dụng để tạo client và trao đổi trực tiếp với gemini qua API, tạo môi trường tương tác cho user. File còn lại là gemini_functions.py, chứa các hàm cần thiết cho việc giao tiếp ở file main.
- Các chức năng đã chạy thành công bao gồm:
 - o Nhận diện giọng nói với độ chính xác tương đối cao, ước tính khoảng trên 90% trong môi trường không có quá nhiều nhiễu.
 - o Kết nối thành công với Gemini và nhận được câu trả lời
 - o Câu trả lời được xuất dưới cả dạng text và âm thanh

```

gemini_functions.py
10 from google.cloud import texttospeech
11 import simpleaudio as sa
12
13 # Hàm khởi tạo client, lấy api key từ biến env
14 def get_gemini_client():
15     if 'GEMINI_API_KEY' not in os.environ:
16         raise ValueError("GEMINI_API_KEY chưa được thiết lập. Vui lòng thiết lập biến môi trường này.")
17     return genai.Client()
18
19 # Hàm khởi tạo đoạn chat có bộ nhớ
20 speak_text() except Exception as e

```

```

chatbot_main.py
10 chatbot = Chatbot()
11 while True:
12     text = input("Bạn: ")
13     if text.strip() == "":
14         continue
15     response = chatbot.get_response(text)
16     print("Chatbot: ", response.text)
17     audio = response.audio
18     if audio:
19         play(audio)
20
21 # Khởi tạo client Gemini
22 client = get_gemini_client()
23
24 # Khởi tạo đoạn chat có bộ nhớ
25 chat_session = chatbot.create_chat_session(client)
26
27 # Bắt đầu tương tác
28 while True:
29     text = input("Bạn: ")
30     if text.strip() == "":
31         continue
32     response = chat_session.send_message(text)
33     print("Chatbot: ", response.text)
34     audio = response.audio
35     if audio:
36         play(audio)

```

Run: chatbot_main

```

Tuy nhiên, nếu xét về độ phổ biến toàn cầu, doanh số bán hàng và độ nhận diện thương hiệu rộng khắp, **Toyota** thường được coi là hãng xe nổi tiếng hàng đầu. Họ nổi tiếng với
Bot: Đang lắng nghe...
Bạn: một cộng một bằng mấy
Chatbot: Một cộng một bằng hai.
Bot: Đang lắng nghe...
Bạn: xe leo dốc thì nên đi số mấy
Chatbot: Việc đi số mấy khi leo dốc phụ thuộc vào **độ dốc của con dốc và loại xe (số sàn hay số tự động)**. Tuy nhiên, nguyên tắc chung là: **nên đi số thấp**.

### Đối với xe số sàn (Manual Transmission):

* **Dốc nhẹ hoặc trung bình:** Có thể đi số 2 hoặc số 3.
* **Dốc cao, gập hoặc xe đang có tải nặng:** Nên đi số 1 hoặc số 2.
* **Nguyên tắc:** Khi cảm thấy xe bắt đầu 1, máy yếu đi, hãy về số thấp hơn ngay lập tức. Số thấp giúp động cơ có đủ lực kéo (mô-men xoắn) để đẩy xe lên dốc mà không bị quá tải.

### Đối với xe số tự động (Automatic Transmission):

* **Không nên để ở chế độ "D" (Drive) hoàn toàn** khi leo dốc dài vì hộp số sẽ có xu hướng tự động lên số cao để tiết kiệm nhiên liệu, làm xe mất lực kéo.

```

2.3 Điểm hạn chế hiện tại

- Kiến thức và thông tin nhận được bị giới hạn bởi gemini, do bị cut of từ tháng 1/2025.
- Chưa cài các yêu cầu mặc định cho chatbot, ví dụ trả lời ngắn gọn, đúng trọng tâm, theo phong cách nào đó,... dẫn đến nhiều lúc câu trả lời dài, nghe tốn thời gian
- Chưa kết nối được với công cụ tìm kiếm của google để tra cứu thông tin thời gian thực.
- Chưa đa dạng các API sử dụng, có thể cân nhắc các api về thời tiết, vị trí
- Chưa có lớp xử lý lỗi hay biện pháp chạy offline tạm thời

3. Kế hoạch thời gian tới

- Tra cứu thời gian thực (Real-time Grounding): Cấu hình Gemini Function Calling để sử dụng Google Search làm công cụ tìm kiếm, khắc phục giới hạn kiến thức.
- Thiết lập tính cách (Persona): Thêm system_instruction vào cấu hình chat để Gemini trả lời ngắn gọn, đúng trọng tâm và theo phong cách trợ lý xe hơi.
- Mở rộng chức năng API: Mở rộng Function Calling để tích hợp các API dịch vụ bên thứ ba (ví dụ: thời tiết, vị trí).
- Triển khai thử nghiệm trên PI5.
- Triển khai logic bắt lỗi mạng (APIError) và hệ thống dự phòng cục bộ (local fallback) cho các lệnh đơn giản khi mất kết nối Cloud.

4. Phụ lục code

Do các API liên quan đến tài khoản trả phí nên sẽ không public lên github nên code sẽ để trong phụ lục

4.1 File gemini_functions.py

```
# File funtions này để chứa các hàm giao tiếp API và xử lý giọng nói/âm thanh, sửa đổi và phát triển các chức năng sẽ được làm chủ yếu trong file này

# Thư viện chung cho hệ thống và thiết lập thiết bị, môi trường

import os

import speech_recognition as sr

from google import genai

from google.genai.errors import APIError


# Thư viện cho Google Cloud Speech-to-Text và Text-to-Speech

from google.cloud import speech

from google.cloud import texttospeech

import simpleaudio as sa


# Hàm khởi tạo client, lấy api key từ biến env

def get_gemini_client():

    if 'GEMINI_API_KEY' not in os.environ:

        raise ValueError("GEMINI_API_KEY chưa được thiết lập. Vui lòng thiết lập biến môi trường này.")

    return genai.Client()


# Hàm khởi tạo đoạn chat có bộ nhớ

def create_chat_session(client: genai.Client):

    print("Đang khởi tạo Chat...")

    chat = client.chats.create(model="gemini-2.5-flash")

    return chat


# Hàm STT (voice thành text)

def listen_and_recognize() -> str | None:

    r = sr.Recognizer()
```

```
with sr.Microphone() as source:

    print("\nBot: Đang lắng nghe...")

    r.adjust_for_ambient_noise(source, duration=0.5)

    try:

        audio = r.listen(source)

    except sr.WaitTimeoutError:

        return None

# Chuyển đổi âm thanh thành dữ liệu (LINEAR16, 16000Hz)

try:

    wav_data = audio.get_wav_data(convert_rate=16000)

except Exception:

    return None

# Sử dụng Google Cloud Speech-to-Text (tự động xác thực qua JSON Key)

try:

    client = speech.SpeechClient()

    audio_config = speech.RecognitionConfig(

        encoding=speech.RecognitionConfig.AudioEncoding.LINEAR16,

        sample_rate_hertz=16000,

        language_code='vi-VN',

    )

    recognition_audio = speech.RecognitionAudio(content=wav_data)

    response = client.recognize(config=audio_config, audio=recognition_audio)

    if response.results:

        text = response.results[0].alternatives[0].transcript
```

```
        print(f"Bạn: {text}")

        return text

    return None

except Exception as e:

    print(f"\n[LỖI CLOUD STT]: {e}")

    print("Kiểm tra biến môi trường GOOGLE_APPLICATION_CREDENTIALS.")

    return None

# Hàm TTS (text từ gemini ra voice)
def speak_text(text_to_speak: str):

    try:

        client = texttospeech.TextToSpeechClient()

        synthesis_input = texttospeech.SynthesisInput(text=text_to_speak)

        voice = texttospeech.VoiceSelectionParams(

            language_code="vi-VN",

            name="vi-VN-Standard-A" # Giọng nói tiêu chuẩn tiếng Việt

        )

        audio_config = texttospeech.AudioConfig(

            audio_encoding=texttospeech.AudioEncoding.LINEAR16,

            sample_rate_hertz=24000 # Quan trọng: Cloud TTS xuất 24kHz

        )

        response = client.synthesize_speech(

            input=synthesis_input, voice=voice, audio_config=audio_config

        )
```

```
# Phát âm thanh bằng simpleaudio

play_obj = sa.play_buffer(
    response.audio_content,
    num_channels=1,
    bytes_per_sample=2, # LINEAR16 = 16 bit = 2 bytes
    sample_rate=24000
)

play_obj.wait_done()

except Exception as e:

    print(f"\n[LỖI PHÁT ÂM THANH TTS]: {e}")
```

4.2 File chatbot_main.py

```
# chatbot_main.py - Tương tác giọng nói cốt lõi

import gemini_functions as gf

from google.genai.errors import APIError

def run_chatbot_interface():

    # Khởi tạo Client và Chat Session

    try:

        client = gf.get_gemini_client()

        chat = gf.create_chat_session(client)

    except Exception as e:

        print(f"Lỗi khởi tạo: {e}. Vui lòng kiểm tra API Key và thư viện.")

        return

    print("\n-Nói 'bye bye' để kết thúc.")

    while True:

        # B1: Lắng nghe và chuyển giọng nói thành văn bản (STT)
```

```
user_input = gf.listen_and_recognize()

if user_input is None:

    gf.speak_text("Xin lỗi, tôi không nghe rõ. Bạn có thể nói lại không?")

    continue


if user_input.lower() in ['bye bye', 'tạm biệt']:

    gf.speak_text("Tạm biệt! Hẹn gặp lại.")

    print("\nKết thúc.")

    break


# B2: Gửi tin nhắn và nhận phản hồi (Stream)

try:

    response_stream = chat.send_message_stream(user_input)

    full_response_text = ""

    print("Chatbot:", end=" ", flush=True)

    # Thu thập toàn bộ phản hồi

    for chunk in response_stream:

        print(chunk.text, end="", flush=True)

        full_response_text += chunk.text

    print() # Xuống dòng

    # B3: Chuyển văn bản phản hồi thành giọng nói và phát (TTS)

    if full_response_text:

        gf.speak_text(full_response_text)


except APIError as e:
```



```
        print(f"\n[LỖI API]: Đã xảy ra sự cố máy chủ. Chi tiết: {e}")

        gf.speak_text("Đã xảy ra lỗi kết nối. Vui lòng kiểm tra API Key của Gemini.")

    except Exception as e:

        print(f"\n[LỖI]: {e}")

        gf.speak_text("Đã xảy ra lỗi không xác định trong quá trình xử lý.")

if __name__ == "__main__":

    run_chatbot_interface()
```